FlashNorm: Fast Normalization for Transformers

Anonymous Author(s)
Affiliation
Address
email

Abstract

Normalization layers are ubiquitous in large language models (LLMs) yet represent a compute bottleneck: on hardware with distinct vector and matrix execution units, the RMS calculation blocks the subsequent matrix multiplication, preventing parallel execution. We present **FlashNorm**, an *exact* reformulation of RMSNorm followed by a linear layer that (i) eliminates the normalization weights by folding them into the subsequent linear layer, and (ii) defers the scalar RMS normalization to the output of the matrix multiplication, enabling the two operations to execute in parallel. The same techniques extend to LayerNorm and Dynamic Tanh (DyT). And due to RMS scale invariance, a linear layer between two RMSNorms makes the first one redundant, removing the pre-attention RMSNorm in QKV-normalized models (e.g., Gemma 4) and in MLA-models with latent normalization (e.g., DeepSeek-V2, Mistral Small 4). On an NVIDIA T4 GPU, FlashNorm achieves **33–35%** lower latency on the norm-then-project operation in the compute-bound (prefill) regime at SmolLM2-135M scale, and **12–14%** at Llama-7B scale. On an NVIDIA A100, FlashNorm achieves a **13%** end-to-end speedup at decode on Llama-3.2-1B in HuggingFace Transformers. We verify zero-loss weight folding on three models. Beyond inference speed, FlashNorm simplifies model implementations by reducing parameter tensor count.

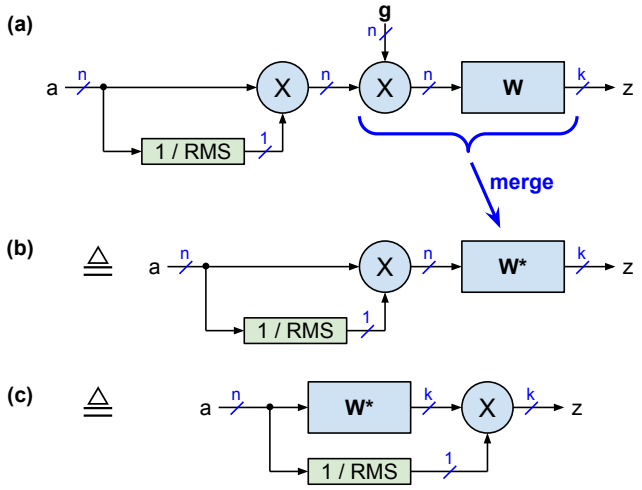

Figure 1: Mathematically identical implementations of RMSNorm followed by a linear layer. (a) Standard: normalization weights $\mathbf{g}$ are applied element-wise before $\mathbf{W}$. (b) **Weightless**: $\mathbf{g}$ is folded into $\mathbf{W}^* = \mathrm{diag}(\mathbf{g})\mathbf{W}$. (c) **FlashNorm**: normalization is deferred to the output, enabling matrix multiplication ($\mathbf{W}^*$) and RMS calculation to execute in parallel. $\triangleq$ denotes mathematical identity.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

1 Introduction

Normalization is indispensable to the training stability and generalization of modern LLMs. RMSNorm [1], used by Llama [2], Gemma [3], Mistral [4], and OLMo 2 [5], normalizes each activation vector by its root mean square (RMS) before scaling by learned weights. In the standard transformer network [6], RMSNorm is followed immediately by a linear (projection) layer—in both the attention and feed-forward sub-networks of every transformer block.

This sequential structure conceals a hardware inefficiency. On processors with dedicated vector units (for element-wise ops) and matrix units (for matrix multiplication), the RMS value must be fully computed *before* the linear layer can begin, because the normalized activations are its input. The matrix unit therefore sits idle during the RMS calculation, a bottleneck that compounds across all transformer layers and inference calls.

We address this bottleneck with **FlashNorm**, which makes two mathematically equivalent transformations to the RMSNorm-then-project pipeline shown in Fig. 1(a):

1. **Weightless normalization.** The per-channel normalization weights $\mathbf{g}$ are absorbed into the weight matrix of the following linear layer, eliminating them as a separate parameter tensor as illustrated in Fig. 1(b).
2. **Deferred normalization.** Because scaling by $1/\mathrm{RMS}(\mathbf{a})$ commutes with matrix multiplication, the scalar normalization is moved to the *output* of the linear layer as shown in Fig. 1(c). This allows the matrix multiplication and the RMS calculation to execute in parallel on separate hardware units.

The name is inspired by FlashAttention [7], where the softmax denominator is similarly deferred past a matrix multiplication to enable parallel processing of keys and values.

FlashNorm requires no retraining, and weight folding can be applied post-hoc to any pretrained checkpoint. We further show how the same ideas extend to Layer Normalization [8], DyT [9], GLU-variant FFNs [10], and RoPE-based attention [11].

Contributions.

- We formally characterize the commutativity of RMSNorm with matrix multiplication (Proposition 2) and derive the exact conditions under which deferred normalization holds.
- We show that RMS scale invariance allows an RMSNorm–linear–RMSNorm sequence to drop the first RMSNorm entirely (Proposition 3), eliminating the pre-attention RMSNorm in models with QKV-normalization [12] such as Gemma 4 [13] and in MLA-models with latent normalization (e.g., DeepSeek-V2 [14], Mistral Small 4 [15], OpenMythos [16]).
- We present FlashNorm as a unified framework covering RMSNorm, LayerNorm, DyT, GLU-FFNs, and attention with RoPE.
- We verify zero-loss weight-folding on SmolLM2-135M, Llama-3.2-1B, and Llama-3.1-8B across wikitext, MMLU, and HellaSwag (Table 1), and report operation-level speedups on T4, A100, and H100.
- And we identify the kernel-engineering work required to extend to compute-bound regimes and provide a reference design.

2 Background and Related Work

Normalization in transformers. Layer Normalization [8] applies mean centering followed by variance normalization with learned scale and bias. RMSNorm [1] drops mean centering and bias, reducing to pure RMS-based scaling; it has become the dominant normalization choice in recent open LLMs due to its simplicity and comparable empirical performance. Dynamic Tanh (DyT) [9] is a recently proposed drop-in replacement that eliminates the normalization computation entirely, replacing it with a parameterized $\tanh$; FlashNorm is complementary in that it targets the interaction between normalization and the subsequent linear layer.

Kernel fusion and operation reordering. Fusing small operations into a single GPU kernel is a standard optimization to reduce memory bandwidth and kernel-launch overhead. Mehta et al. [17] report that modifications to the layer normalization implementation significantly affect tokens-per-second throughput, attributing this to a lack of kernel fusion. FlashNorm reduces the number of kernel launches by eliminating the normalization weight application as a separate step, and additionally enables parallelism between the vector and matrix units that kernel fusion alone cannot achieve.

Weight folding and parameter elimination. The idea of absorbing one layer's parameters into a neighboring layer appears in several forms. Batch normalization folding into convolutional weights is standard practice in quantization-aware training and model deployment [18, 19] and `torch.compile` can perform it automatically [20]. OLMo 1 models use weightless LayerNorm, which they call *non-parametric* [5]. FlashNorm applies analogous reasoning to normalization weights of RMSNorm and LayerNorm.

FlashAttention. FlashAttention [7] is the closest conceptual predecessor: it defers the softmax normalization (division by $\sum \exp$) past the multiplication with the value matrix $\mathbf{V}$, enabling online computation and improving memory efficiency. FlashNorm applies the same reordering insight to the simpler setting of RMSNorm and a dense linear layer.

3 FlashNorm

We now develop FlashNorm formally. All propositions assume bias-free linear layers unless stated otherwise; Section 3.4 addresses the biased case.

3.1 Weightless Normalization

Let $\mathbf{a} \in \mathbb{R}^n$ be an activation vector. RMSNorm computes

$$\mathrm{RMSNorm}(\mathbf{a})_i = \frac{a_i}{\mathrm{RMS}(\mathbf{a})} \cdot g_i, \qquad \mathrm{RMS}(\mathbf{a}) = \sqrt{\frac{1}{n}\sum_{i=1}^{n} a_i^2},$$

where $\mathbf{g} \in \mathbb{R}^n$ are learned normalization weights. In transformer models, RMSNorm is followed by a linear layer $\mathbf{W} \in \mathbb{R}^{n\times k}$ as illustrated in Fig. 1(a):

$$\mathbf{z} = \mathrm{RMSNorm}(\mathbf{a})\,\mathbf{W}.$$

Proposition 1 (Weightless normalization)**.** *Let $\mathbf{W}^* \in \mathbb{R}^{n\times k}$ be defined by $W^*_{i,j} = g_i \cdot W_{i,j}$. Then $\mathbf{z} = \frac{\mathbf{a}}{\mathrm{RMS}(\mathbf{a})}\,\mathbf{W}^*$, i.e., the normalization weights $\mathbf{g}$ can be absorbed into $\mathbf{W}$ and need not be stored or applied separately.*

Proof. $z_j = \sum_i \frac{a_i}{\mathrm{RMS}(\mathbf{a})} g_i W_{i,j} = \sum_i \frac{a_i}{\mathrm{RMS}(\mathbf{a})} W^*_{i,j}$. □

This proposition holds for linear layers with or without a bias term at the output. The practical implication is that the normalization weight vector $\mathbf{g}$ becomes a redundant parameter: it can be folded into $\mathbf{W}$ once at load time, saving memory, reducing the parameter count, and eliminating one vector-multiply kernel call per forward pass.

3.2 Deferred Normalization

Proposition 2 (Commutativity of RMS scaling with matrix multiplication)**.** *Let $\mathbf{W}^*$ be as in Proposition 1 and let the linear layer be bias-free. Then*

$$\mathbf{z} = \frac{\mathbf{a}}{\mathrm{RMS}(\mathbf{a})}\,\mathbf{W}^* = (\mathbf{a}\,\mathbf{W}^*) \cdot \frac{1}{\mathrm{RMS}(\mathbf{a})}.$$

That is, the scalar normalization $1/\mathrm{RMS}(\mathbf{a})$ can be applied after *the matrix multiplication.*

Proof. Scalar multiplication commutes with matrix multiplication: $(\alpha\mathbf{a})\mathbf{W}^* = (\mathbf{a}\mathbf{W}^*)\alpha$ for any scalar α. Setting $\alpha = 1/\mathrm{RMS}(\mathbf{a})$ gives the result. □

Corollary 1 (Parallel execution)**.** *Under the conditions of Proposition 2, computing* $\mathbf{a}\mathbf{W}^*$ *and* $\mathrm{RMS}(\mathbf{a})$ *are* independent *computations that may proceed in parallel. The final result requires only a single vector-scalar multiply after both complete.*

Remark 1 (Bias case)**.** *If the linear layer has a bias* $\mathbf{c} \in \mathbb{R}^k$*, deferred normalization does not hold directly because* $(\mathbf{a}\mathbf{W}^* + \mathbf{c})/\mathrm{RMS}(\mathbf{a}) \neq \mathbf{a}\mathbf{W}^*/\mathrm{RMS}(\mathbf{a}) + \mathbf{c}$*. In this case, Proposition 1 (weightless normalization) still applies, but the normalization must precede the bias addition.*

Remark 2 (Unfolded case)**.** *Proposition 2 requires only that* $1/\mathrm{RMS}(\mathbf{a})$ *be a per-token scalar. Propositions 1 and 2 are therefore independent: a framework implementing deferred normalization alone obtains the parallel-execution and fused-kernel speedups (Corollary 1 and Appendix G) for any existing RMSNorm checkpoint, with no weight-folding required.*

Figure 1 illustrates the three mathematically equivalent implementations: (a) standard RMSNorm-then-project, (b) weightless normalization, and (c) the full FlashNorm with deferred normalization.

3.3 Cancellation of Pre-Normalization

Proposition 3 (RMSNorm cancellation)**.** *For activation* $\mathbf{a}$*, linear layer* $\mathbf{W}^*$ *(with normalization weights already folded in as per Proposition 1), and a second RMSNorm applied to the output,*

$$\mathrm{RMSNorm}\left(\frac{\mathbf{a}}{\mathrm{RMS}(\mathbf{a})}\,\mathbf{W}^*\right) = \mathrm{RMSNorm}(\mathbf{a}\,\mathbf{W}^*)\,,$$

i.e., the first RMSNorm is redundant and can be eliminated entirely as shown in Fig. 2.

Proof. Let $\mathbf{b} = \mathbf{a}\mathbf{W}^*$, $s_a = 1/\mathrm{RMS}(\mathbf{a})$, and $\mathbf{c} = \mathbf{b} \cdot s_a$, see Fig. 2. By scale invariance of RMS, $\mathrm{RMS}(\mathbf{c}) = s_a \cdot \mathrm{RMS}(\mathbf{b})$, so the product of the two scalers s_a and $\frac{1}{\mathrm{RMS}(\mathbf{c})}$ is $s_a \cdot \frac{1}{\mathrm{RMS}(\mathbf{c})} = s_a \cdot \frac{1}{s_a \cdot \mathrm{RMS}(\mathbf{b})} = \frac{1}{\mathrm{RMS}(\mathbf{b})}$, and therefore $\mathrm{RMSNorm}(\mathbf{c}) = \mathrm{RMSNorm}(\mathbf{b})$. □

Eliminating pre-attention RMSNorm with QKV-normalization. Some models such as Gemma 4 [13] apply QKV-normalization [12] to queries, keys, *and values*. The sequence is: pre-attention RMSNorm → QKV-projection → QKV-RMSNorm. By Proposition 3, the pre-attention RMSNorm cancels and can be dropped entirely, saving one full RMSNorm per each transformer block.

Eliminating pre-attention RMSNorm with MLA latent normalization. The sequence for MLA with latent normalization is: pre-attention RMSNorm → down-projection into latent space → latent RMSNorm, which matches the pattern of Proposition 3 exactly. The pre-attention RMSNorm therefore cancels and can be eliminated. This optimization applies to all the MLA models we inspected, including DeepSeek-V2 [14], Mistral Small 4 [15], and OpenMythos [16].

Epsilon caveat. The scale-invariance $\mathrm{RMS}(s \cdot \mathbf{a}) = s \cdot \mathrm{RMS}(\mathbf{a})$ does not hold exactly for the ε-regularized variant $\mathrm{RMS}_\varepsilon(\mathbf{a}) = \sqrt{\varepsilon + \frac{1}{n}\sum a_i^2}$. The discrepancy is negligible in practice: ε only has material effect when the activation energy is near zero, where it limits up-scaling, see appendix.

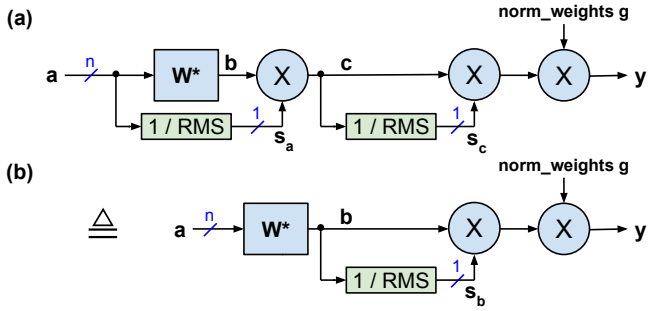

Figure 2: Weightless RMSNorm followed by a linear layer and a second RMSNorm: (a) unoptimized version; (b) optimized version where the first RMSNorm is eliminated by Proposition 3.

3.4 Support for Normalization Bias

LayerNorm [8] and DyT [9] include an additive bias $\boldsymbol{\beta}$ after scaling by $\mathbf{g}$. Fig. 3 illustrates how this bias can be moved to the output of the following linear layer, resulting in a modified bias $\mathbf{c}^* = \mathbf{c} + \boldsymbol{\beta}\mathbf{W}$, see Fig. 3(b). After this elimination of $\boldsymbol{\beta}$, the normalization weights $\mathbf{g}$ can be folded into the linear layer as per Proposition 1. Note that deferred normalization (Proposition 2) does not apply when a bias is present; the scalar $1/\mathrm{RMS}(\mathbf{a})$ must be applied before adding $\mathbf{c}^*$, which still allows for parallel execution of matrix multiplication and RMS calculation.

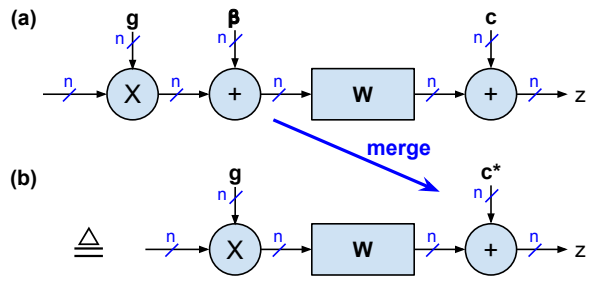

Figure 3: Elimination of bias $\boldsymbol{\beta}$: (a) Before elimination with $\boldsymbol{\beta}$ between normalization weights $\mathbf{g}$ and linear layer. (b) Optimized version with new bias term $\mathbf{c}^* = \mathbf{c} + \boldsymbol{\beta}\mathbf{W}$ at the output.

3.5 Folding Mean Centering into a Preceding Linear Layer

LayerNorm applies mean centering before RMSNorm. If mean centering is preceded by a linear layer with weights $\mathbf{V} \in \mathbb{R}^{n \times n}$, the centering can be absorbed into $\mathbf{V}$ without retraining, see Fig. 4: The mean μ is calculated from the linear layer outputs y_j as $\mu = \frac{1}{n}\sum_j y_j$ where $y_j = \sum_i x_i V_{i,j}$. Let $s_i = \sum_j V_{i,j}$ be the row sum of row i, we can then calculate μ directly from the inputs x_i as

$$\mu = \frac{1}{n}\sum_{j=1}^{n}\sum_{i=1}^{n} x_i V_{i,j} = \frac{1}{n}\sum_{i=1}^{n} x_i \left[\sum_{j=1}^{n} V_{i,j}\right] = \frac{1}{n}\sum_{i=1}^{n} x_i s_i.$$

The mean-centered outputs are now

$$a_j = y_j - \mu = \sum_{i=1}^{n} x_i V_{i,j} - \frac{1}{n}\sum_{i=1}^{n} x_i s_i = \sum_{i=1}^{n} x_i \underbrace{\left(V_{i,j} - \frac{s_i}{n}\right)}_{V^*_{i,j}},$$

so the modified weights $V^*_{i,j} = V_{i,j} - s_i/n$ absorb the mean centering entirely, see Fig. 4(b). This also provides a lossless recipe for converting a LayerNorm model to RMSNorm without any retraining.

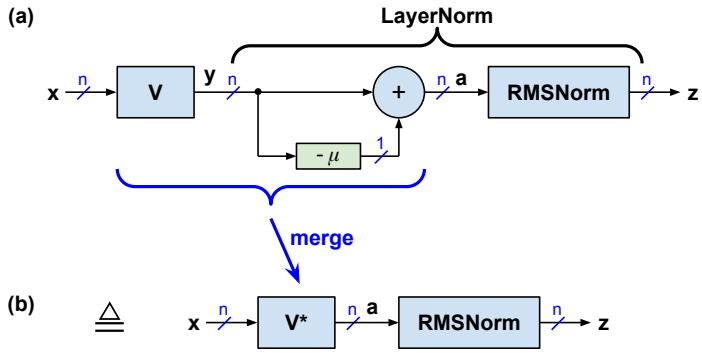

Figure 4: Elimination of mean centering: (a) Original weight matrix $\mathbf{V}$ followed by mean centering. (b) Optimized version where the mean centering is merged into the modified weight matrix $\mathbf{V}^*$.

4 Extensions

4.1 FlashNorm for FFNs with GLU Variants

For GLU-variant FFNs [10], the FlashNorm at the FFN input requires scaling at both the Gate and Up projection outputs, see Fig. 5(a). One of these scaling operations can be deferred to the FFN output, saving $f - n$ multiply operations as shown in Fig. 5(b). See appendix for FFNs with ReLU, ReGLU and Bilinear GLU variants.

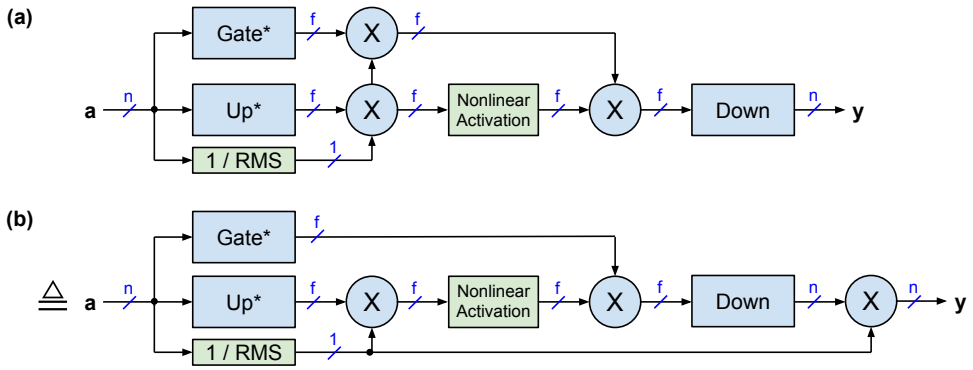

Figure 5: FFN with GLU variant and preceding FlashNorm: (a) unoptimized version; (b) optimized version with fewer scaling multipliers.

4.2 FlashNorm for Attention with RoPE

For the query ($\mathbf{Q}^*$) and key ($\mathbf{K}^*$) projections in RoPE-based attention [11], the $1/\mathrm{RMS}(\mathbf{a})$ factor can be fused with the precomputed RoPE cosine and sine tables, which are shared across all attention heads as shown in Fig. 6. Specifically, replacing $\cos(\cdot)$ with $\cos(\cdot)/\mathrm{RMS}(\mathbf{a})$ (and similarly for $\sin$) eliminates the per-head scaling entirely, saving $2hH - h$ multiply operations, where h is the head dimension and H is the number of heads. Additionally, the $1/\sqrt{h}$ scaled dot-product factor can be folded into the same scaling operation.

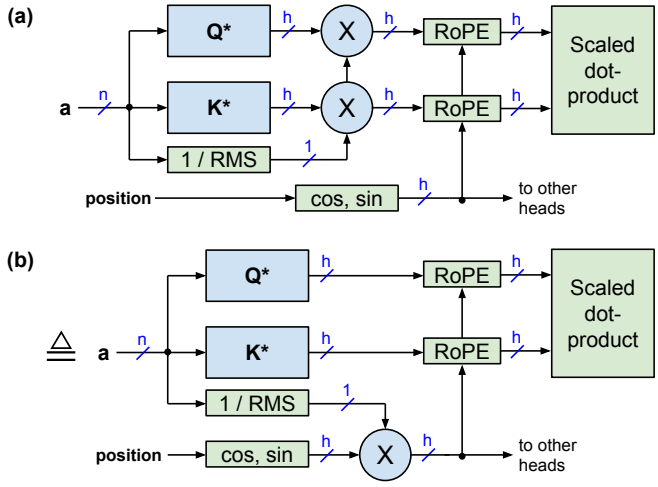

Figure 6: FlashNorm for scaled dot-product attention with RoPE: (a) unoptimized version; (b) optimized version where the normalization is fused with $\cos(\cdot)$ and $\sin(\cdot)$.

4.3 QK-Normalization

Gemma 3 [21], OLMo 2 [5], OpenELM [17] and other LLMs use QK-normalization [22], which applies RMSNorm to queries and keys *after* the projection. Unlike QKV-normalization, V is not normalized, so the pre-attention RMSNorm cannot be fully eliminated—but the $1/\mathrm{RMS}(\mathbf{a})$ scalers for Q and K can still be dropped via Proposition 3 (Fig. 7). Two further optimizations are available:

Fusing QK-norm weights with RoPE. If QK-norm weights are shared across all heads of a layer (as in Gemma 3 and OpenELM), they can be fused with the RoPE cosine and sine vectors, which are already shared across heads, see Fig. 7. The fused tables are computed once per layer, and the normalization weights require only a permutation to align with RoPE's interleaved layout.

Weight-fusion for NoPE channels. Several architectures employ partial RoPE or interleave RoPE and NoPE layers (e.g., Llama 4 [23]). For these models, we can fuse the Q and K normalization weights for the NoPE channels as $g_{\mathbf{QK},i} = g_{\mathbf{Q},i} * g_{\mathbf{K},i}$ for $i \in [\mathrm{NoPE}]$.

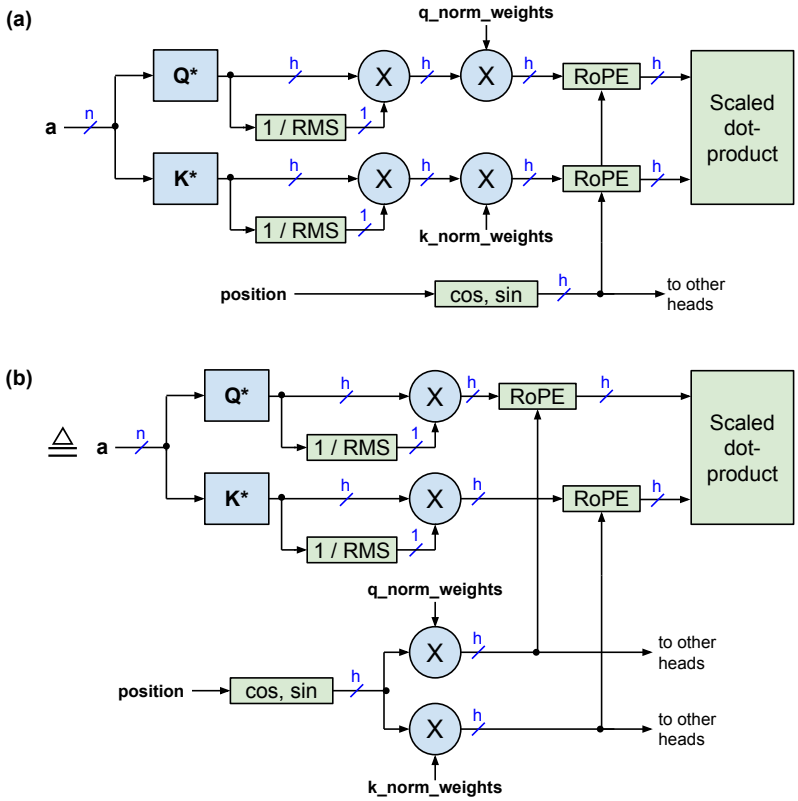

Figure 7: QK-norm with RoPE: (a) unoptimized version; (b) optimized version.

5 Hardware Analysis

We analyze the compute bottleneck that FlashNorm eliminates. Consider a processor with one vector unit (throughput: m ops/cycle) and one matrix unit (throughput: m^2 ops/cycle) where m is the processor width. Specifically:

- Multiplying an n-vector with an $n \times n$ matrix takes n^2 MAD (multiply-add) operations, which takes n^2/m^2 matrix cycles.
- Computing $1/\mathrm{RMS}(\mathbf{a})$ requires n MAD operations (for squaring and adding) plus 2 scalar ops (for $\sqrt{n/x}$), which takes n/m vector cycles if we ignore the 2 scalar ops.
- Scaling an n-vector by a scalar takes n/m vector cycles.

Fig. 8 shows timing diagrams for the example $n = 512$ and $m = 128$. Without FlashNorm (sequential execution), the matrix unit cannot begin until both the RMS and the scaling are complete, imposing an 8 cycle bubble as shown in Fig. 8(a). The subsequent vector-matrix multiply takes $(512)^2/(128)^2 = 16$ cycles; the total is 24 cycles. With row-major ordering, the first rows of $\mathbf{W}^*$ can be processed while later rows of the RMS accumulation are still in flight, reducing the bubble to 5 cycles (total: 21 cycles), but the bottleneck persists, see Fig. 8(b). With FlashNorm, the vector-matrix multiply ($\mathbf{aW}^*$) and the RMS calculation proceed fully in parallel on separate units as shown in Fig. 8(c). The scaling at the end can be performed in parallel to the matrix unit if $\mathbf{W}^*$ is processed in column-major order. Total execution: 17 cycles—a **29% reduction** versus the sequential baseline.

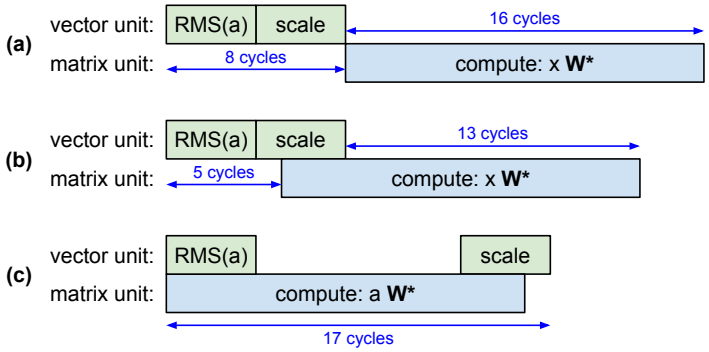

Figure 8: Exemplary timing diagrams. (a) Sequential: matrix unit idles 8 cycles. (b) Partial overlap with interleaved scaling and vector-matrix multiplication. (c) **FlashNorm**: vector-matrix multiply and RMS run fully in parallel.

6 Experiments

We verified lossless weight folding on SmolLM2-135M [24], Llama-3.2-1B, and Llama-3.1-8B [2]. On SmolLM2-135M under HuggingFace Transformers at fp16, the original and weight-folded models produce identical logits (max difference = 0.0, cosine similarity = 1.0) and identical generated text; at Llama-3.2-1B and Llama-3.1-8B scales under fp32 inference, greedy generation is likewise equivalent to the source. Table 1 reports source-vs-flashified deltas across three model scales and three standard benchmarks (wikitext word perplexity, MMLU 5-shot accuracy, HellaSwag zero-shot accuracy): every delta is within benchmark noise, empirically consistent with the mathematical exactness of Propositions 1 and 2.

Table 1: Quality preservation across model scales and standard benchmarks, via `lm-evaluation-harness` with fp16 inference on an NVIDIA A100 (wikitext at 2048-token context, MMLU at 4096-token context for 5-shot prompts, HellaSwag at 2048-token context). Deltas between source and FlashNorm-folded checkpoints are within benchmark noise at every row, empirically consistent with the mathematical exactness of Propositions 1 and 2.

Model	Benchmark	Source	FlashNorm	Δ
SmolLM2-135M	wikitext word_ppl	23.1262	23.1279	+0.001689
Llama-3.2-1B	wikitext word_ppl	12.8837	12.8842	+0.000497
Llama-3.1-8B	wikitext word_ppl	8.0668	8.0672	+0.000382
Llama-3.1-8B	MMLU acc (5-shot)	0.6556	0.6562	+0.0006
Llama-3.1-8B	HellaSwag acc (0-shot)	0.6071	0.6072	+0.0001

To validate the parallel execution described in Fig. 8, we benchmarked the norm-then-project operation on an NVIDIA T4 GPU (65 TFLOPS FP16, 320 GB/s). We compare sequential execution (RMSNorm $\rightarrow$ GEMM) against FlashNorm with adaptive dispatch: when the GEMM is compute-bound (many tokens), the GEMM runs on tensor cores and $1/\mathrm{RMS}(\mathbf{a})$ runs on vector cores in parallel via separate CUDA streams; when memory-bound (few tokens), FlashNorm falls back to sequential execution to avoid stream overhead. Table 2 shows the results at SmolLM2-135M scale, and Table 3 at Llama-7B scale.

Table 2: Norm-then-project latency on T4 GPU at SmolLM2-135M scale (n=576, k=960). Below the compute-bound threshold, FlashNorm falls back to sequential execution (0% overhead).

Tokens	Sequential	FlashNorm	Speedup	Path
1	0.374 ms	0.374 ms	0.0%	sequential (fallback)
64	0.506 ms	0.506 ms	0.0%	sequential (fallback)
4096	0.704 ms	0.468 ms	+33.6%	parallel (overlap)
8192	0.929 ms	0.599 ms	+35.5%	parallel (overlap)

FlashNorm uses adaptive dispatch: the parallel path is used only when the GEMM is compute-bound (token count exceeds a hardware-dependent threshold). For memory-bound workloads such as autoregressive decoding, FlashNorm falls back to sequential execution (0% overhead). This is because managing parallel CUDA streams introduces scheduling overhead that exceeds the time saved when the GEMM is small. At autoregressive decode on `Llama-3.2-1B-Instruct` in HuggingFace Transformers, RMSNorm operations occupy 24.6% of total wall-clock at bf16, an order of magnitude above their FLOP share; the cumulative norm cost is invariant under weight quantization, so the relative impact of FlashNorm grows in deployments that aggressively quantize linears (Appendix F).

7 Limitations and Discussion

Relationship to quantization. FlashNorm is orthogonal to weight quantization. In fact, folding normalization weights into $\mathbf{W}^*$ before quantization can improve quantization quality by absorbing per-channel scale factors directly into the weight distribution, a technique already used in practice (e.g., SmoothQuant [25]). We leave a systematic study of this interaction to future work.

Activation dynamic range. For implementations utilizing low-precision fixed-point activations, FlashNorm's deferred normalization may trigger overflows during matrix multiplication, as noted in [26]. However, the majority of contemporary frameworks employ floating-point activations alongside either fixed-point or floating-point weights, which inherently mitigates this risk.

Simplification as a primary benefit. Beyond raw speed, FlashNorm simplifies model implementations in a manner analogous to RMSNorm's simplification over LayerNorm, or PaLM's removal of linear-layer biases [27]. Fewer parameter tensors mean smaller checkpoint files, simpler loading logic, and reduced risk of implementation bugs.

8 Conclusion

We presented FlashNorm, a mathematically exact reformulation of RMSNorm-then-project that (1) eliminates normalization weights by folding them into the subsequent linear layer, and (2) defers the RMS operation to after the matrix multiplication, enabling parallel execution. We further showed (3) that RMS scale invariance allows an RMSNorm–linear–RMSNorm sequence to drop the first RMSNorm entirely, removing the pre-attention RMSNorm in models with QKV-normalization (e.g., Gemma 4) or MLA latent normalization (e.g., DeepSeek, Mistral, OpenMythos). GPU benchmarks confirm 33–35% operation-level latency reduction in the compute-bound prefill regime, with identical outputs verified on SmolLM2-135M, Llama-3.2-1B, and Llama-3.1-8B. In the memory-bound decode regime, stream management overhead currently negates the speedup, motivating native kernel-level integration as the primary direction for future work.

Future integration targets include vLLM [28], SGLang [29], FlashInfer [30], HuggingFace Transformers [31], llama.cpp [32], whisper.cpp [33], Ollama [34], LM Studio [35], and MLX [36] as well as systematic study of the interaction with weight quantization methods.

References

[1] Biao Zhang and Rico Sennrich. Root mean square layer normalization. October 2019. *arXiv:1910.07467*.

[2] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and efficient foundation language models. February 2023. *arXiv:2302.13971*.

[3] Gemma Team, Google DeepMind. Gemma: Open Models Based on Gemini Research and Technology. 2024.

[4] Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7B. October 2023. *arXiv:2310.06825*.

[5] Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, Nathan Lambert, Dustin Schwenk, Oyvind Tafjord, Taira Anderson, David Atkinson, Faeze Brahman, Christopher Clark, Pradeep Dasigi, Nouha Dziri, Michal Guerquin, Hamish Ivison, Pang Wei Koh, Jiacheng Liu, Saumya Malik, William Merrill, Lester James V Miranda, Jacob Morrison, Tyler Murray, Crystal Nam, Valentina Pyatkin, Aman Rangapur, Michael Schmitz, Sam Skjonsberg, David Wadden, Christopher Wilhelm, Michael Wilson, Luke Zettlemoyer, Ali Farhadi, Noah A Smith, and Hannaneh Hajishirzi. 2 OLMo 2 Furious. December 2024. *arXiv:2501.00656*.

[6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. June 2017. *arXiv:1706.03762*.

[7] Tri Dao, Daniel Y Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. May 2022. *arXiv:2205.14135*.

[8] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer Normalization. July 2016. *arXiv:1607.06450*.

[9] Jiachen Zhu, Xinlei Chen, Kaiming He, Yann LeCun, and Zhuang Liu. Transformers without Normalization. 2025. *arXiv:2503.10622*.

[10] Noam Shazeer. GLU Variants Improve Transformer. February 2020. *arXiv:2002.05202*.

[11] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with Rotary Position Embedding. April 2021. *arXiv:2104.09864*.

[12] Stephen Menary, Samuel Kaski, and Andre Freitas. Transformer normalisation layers and the independence of semantic subspaces. June 2024. *arXiv:2406.17837*.

[13] Clement Farabet and Olivier Lacombe. Gemma 4: Byte for byte, the most capable open models. April 2026.

[14] DeepSeek-AI, Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Dengr, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Hao Yang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jin Chen, Jingyang Yuan, Junjie Qiu, et al. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. 2024. *arXiv:2405.04434*.

[15] Mistral AI. Introducing Mistral Small 4. March 2026.

[16] Kye Gomez. OpenMythos: A Theoretical Reconstruction of the Claude Mythos Architecture, 2026. URL `https://github.com/kyegomez/OpenMythos`.

[17] Sachin Mehta, Mohammad Hossein Sekhavat, Qingqing Cao, Maxwell Horton, Yanzi Jin, Chenfan Sun, Iman Mirzadeh, Mahyar Najibi, Dmitry Belenko, Peter Zatloukal, and Mohammad Rastegari. OpenELM: An efficient language model family with open-source training and inference framework. April 2024. *arXiv:2404.14619*.

[18] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. December 2017. *arXiv:1712.05877*.

[19] Kaichao You, Guo Qin, Anchang Bao, Meng Cao, Ping Huang, Jiulong Shan, and Mingsheng Long. Efficient ConvBN blocks for transfer learning and beyond. May 2023. *arXiv:2305.11624*.

[20] PyTorch Tutorials. Building a Convolution/Batch Norm fuser with torch.compile, 2026.

[21] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, Louis Rouillard, Thomas Mesnard, Geoffrey Cideron, Jean-Bastien Grill, Sabela Ramos, Edouard Yvinec, Michelle Casbon, Etienne Pot, Ivo Penchev, Gaël Liu, Francesco Visin, Kathleen Kenealy, Lucas Beyer, Xiaohai Zhai, Anton Tsitsulin, Robert Busa-Fekete, Alex Feng, Noveen Sachdeva, Benjamin Coleman, Yi Gao, Basil Mustafa, Iain Barr, Emilio Parisotto, David Tian, Matan Eyal, Colin Cherry, Jan-Thorsten Peter, Danila Sinopalnikov, et al. Gemma 3 Technical Report. March 2025. *arXiv:2503.19786*.

[22] Alex Henry, Prudhvi Raj Dachapally, Shubham Pawar, and Yuxuan Chen. Query-key normalization for transformers. October 2020. *arXiv:2010.04245*.

[23] Meta. The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation. 2025.

[24] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Lewis Tunstall, Agustín Piqueres, Andres Marafioti, Cyril Zakka, Leandro von Werra, and Thomas Wolf. SmolLM2 - with great data, comes great performance, 2024. URL https://huggingface.co/HuggingFaceTB/SmolLM2-1.7B.

[25] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. November 2022. *arXiv:2211.10438*.

[26] Callum McLean, Luke Y. Prince, Alexandre Payot, et al. MXNorm: Reusing MXFP block scales for efficient tensor normalisation. March 2026. *arXiv:2603.13180*.

[27] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, et al. PaLM: Scaling language modeling with Pathways. April 2022. *arXiv:2204.02311*.

[28] vLLM Project. vLLM. URL https://github.com/vllm-project/vllm.

[29] SGLang. SGLang. URL https://github.com/sgl-project/sglang.

[30] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. FlashInfer: Efficient and customizable attention engine for LLM inference serving. January 2025. *arXiv:2501.01005*.

[31] HuggingFace. Transformers. URL https://huggingface.co/docs/transformers.

[32] Georgi Gerganov. llama.cpp. . URL https://github.com/ggml-org/llama.cpp.

[33] Georgi Gerganov. whisper.cpp. . URL https://github.com/ggml-org/whisper.cpp.

[34] Ollama. Ollama. URL https://github.com/ollama/ollama.

[35] LM Studio. LM Studio. URL https://lmstudio.ai.

[36] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on Apple silicon. 2023. URL https://github.com/ml-explore/mlx.

[37] Wikipedia. Rectifier (neural networks), 2024. Accessed June-2024.

[38] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient Finetuning of Quantized LLMs. 2023. *arXiv:2305.14314*.

[39] Philippe Tillet, H. T. Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, 2019. URL https://github.com/triton-lang/triton.

A Additional notes

Scope. The mathematics of FlashNorm (Propositions 1 and 2) is exact in both directions. Applying FlashNorm post-hoc to a trained model is lossless: the folded model's outputs are identical to the source under fp32 and within benchmark noise under fp16. Training a new model directly with a FlashNorm-shaped normalization-plus-linear block is equally valid: the FlashNorm parameterization (no explicit $\mathbf{g}$, RMS scalar applied after the matmul) represents the same function class as standard RMSNorm followed by a linear layer, so any function reachable under the standard parameterization is reachable under the FlashNorm parameterization. Experiments in this paper cover the post-hoc application; training-from-scratch studies are left to future work.

The FlashNorm transformation is mathematically exact (Propositions 1 and 2), and under fp32 inference produces identical greedy generation across all three model scales. Under lossy inference precisions (fp16/bf16 in HuggingFace Transformers at larger scales, or vLLM's PagedAttention at any precision), precomputed merged weights can produce a one-token argmax divergence at tight decision points, which downstream greedy decoding then amplifies. The effect is more visible at larger model scales where argmax decisions are tighter. This is a general consequence of precomputing weight-folded tensors for lossy-inference kernels and is not specific to FlashNorm. A native fused `RMSNorm+QKV` kernel that defers $\mathbf{g}$ to runtime, rather than precomputing it into $\mathbf{W}^*$, eliminates the framework dependency and is the primary direction for ongoing work.

Table 3: Norm-then-project latency on T4 GPU at Llama-7B scale (n=4096, k=4096).

Tokens	Sequential	FlashNorm	Speedup	Path
1	0.641 ms	0.641 ms	0.0%	sequential (fallback)
64	0.307 ms	0.307 ms	0.0%	sequential (fallback)
256	0.577 ms	0.577 ms	0.0%	sequential (fallback)
1024	1.882 ms	1.654 ms	+12.1%	parallel (overlap)
4096	7.628 ms	6.570 ms	+13.9%	parallel (overlap)

While the speedup on the norm-then-project operation itself is significant (33–35%), the end-to-end model speedup is modest because this operation is a fraction of total layer time. For comparison, we measured a throughput of 204 tokens per second for OpenELM-270M with 4-bit weight quantization using the MLX framework on an M1 MacBook Air. This throughput increases to only 225 tokens per second when we remove RMSNorm entirely, giving an upper bound of $\leq$ 10% for this model. Native integration into inference frameworks at the kernel level—where stream management overhead is negligible—would allow the 33–35% operation-level improvement to translate more directly into end-to-end gains, particularly for larger models with bigger GEMMs.

A.1 Prototype Fused Kernel on A100

We further validate FlashNorm on an NVIDIA A100 GPU (312 TFLOPS FP16, 2 TB/s HBM) by prototyping a fused Triton kernel that merges `RMSNorm` with the subsequent QKV projection into a single GPU kernel, benchmarked against two production-realistic baselines: PyTorch's fused `torch.nn.functional.rms_norm` followed by cuBLAS matmul, and vLLM's `_custom_ops.rms_norm` [28] followed by cuBLAS matmul. Table 4 shows +15 to +36% speedup across the full token range at SmolLM2-135M scale against both baselines. Appendix G decomposes this speedup into its two independent sources (weight folding and kernel fusion), reports the full benchmark across three model scales, and discusses the interaction with vendor-tuned matmul codegen that determines the regime of applicability.

Because the mathematics is exact in both directions, FlashNorm admits two complementary use cases: as a post-hoc transformation applied to existing trained checkpoints (the setting measured in this paper), and as a default normalization-plus-linear building block adopted from scratch in new model architectures. In the latter case the per-channel weight tensor $\mathbf{g}$ is absent from the model by construction, which reduces parameter count and removes the framework-integration work required to serve post-hoc-transformed checkpoints through existing fused kernels.

A CUTLASS 3.x Sm90 fused `RMSNorm+linear` kernel built on NVIDIA's production `CollectiveBuilder` and Epilogue Visitor Tree templates (Appendix G.5) confirms that the fusion pattern transfers to production-grade Sm90 infrastructure. Against the HuggingFace baseline

Table 4: Fused Triton `RMSNorm+QKV` kernel vs two production-realistic baselines on A100 at SmolLM2-135M scale (n=576, k=960), fp16 inputs. Baselines correspond to two commonly-deployed production inference paths: HuggingFace Transformers (PyTorch rms_norm plus cuBLAS) and vLLM (`_custom_ops.rms_norm` plus cuBLAS).

Tokens	PyTorch + cuBLAS	vLLM + cuBLAS	FlashNorm	vs PyTorch	vs vLLM
1	0.054 ms	0.052 ms	0.040 ms	+24.8%	+22.2%
16	0.056 ms	0.054 ms	0.039 ms	+29.1%	+26.5%
64	0.084 ms	0.080 ms	0.054 ms	+35.8%	+32.6%
4096	0.072 ms	0.066 ms	0.061 ms	+14.5%	+7.7%

the prototype wins on all 18 shapes; against the optimized FlashInfer baseline it wins on 10 of 18 (all SmolLM2 and Llama-3.2-1B decode shapes), with the remaining 8 losses attributable to a single non-specialized tile choice in the prototype. Closing that gap requires shape-specialized tile dispatch and a custom `CollectiveMainloop` that fuses the RMS accumulation into the matmul's existing tile load, both routine kernel-team work inside production inference frameworks.

B FlashNorm for FFNs with ReLU

For bias-free FFNs with ReLU, the scale-invariance $\mathrm{ReLU}(s \cdot \mathbf{a}) = s \cdot \mathrm{ReLU}(\mathbf{a})$ for $s \geq 0$ [37] allows the RMS normalization to be deferred all the way to the FFN output, past both the up-projection and the ReLU nonlinearity as illustrated in Fig. 9(b). If the up-projection expands from n to f channels, this saves $f - n$ multiply operations compared to applying the scaling at the FFN input.

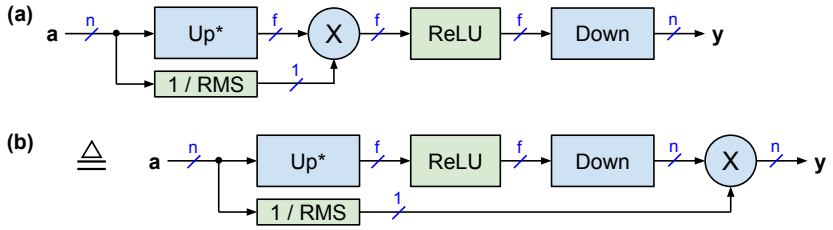

Figure 9: FFN with ReLU and preceding FlashNorm: (a) unoptimized version; (b) optimized version where the normalization is deferred to the output of the FFN. Up and Down denote the linear layers for up and down projections.

C FlashNorm for FFNs with ReGLU and Bilinear GLU

For GLU variants that use the activation function ReLU (ReGLU) or linear (bilinear GLU) [10], both scaling operations after the Gate and Up projection can be consolidated at the output as illustrated in Fig. 10, using the reciprocal of the *squared* RMS, which is the Mean Square $\mathrm{MS}(\mathbf{a})$:

$$\frac{1}{(\mathrm{RMS}(\mathbf{a}))^2} = \frac{1}{\mathrm{MS}(\mathbf{a})} = \frac{n}{\sum_{i=1}^{n} a_i^2}.$$

D RMS with epsilon

Many implementations add a small epsilon ϵ to the RMS value to limit the resulting scaling factor $1/\mathrm{RMS}(\mathbf{a})$ and to avoid division by zero as follows:

$$\mathrm{RMSe}(\vec{a}) = \sqrt{\epsilon + \frac{1}{n}\sum_{i=1}^{n} a_i^2} = \sqrt{\epsilon + (\mathrm{RMS}(\vec{a}))^2}$$

$\mathrm{RMSe}(\vec{a})$ can be used as a drop-in-replacement for RMS. The popular HuggingFace transformer library calls this epsilon `rms_norm_eps`, which is set to 10^{-5} for Llama3.

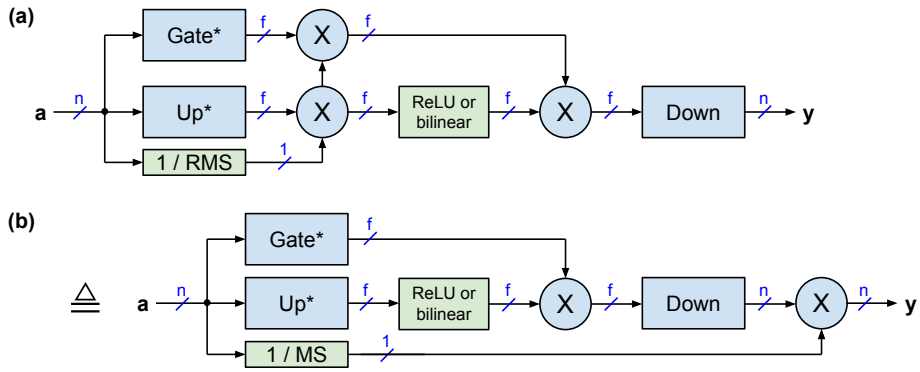

Figure 10: FFN with ReGLU (or bilinear GLU) and preceding FlashNorm: (a) unoptimized version; (b) optimized version, which saves $2f - n$ multiply operations.

E Eliminating $1/n$

This section details a small optimization that eliminates the constant term $1/n$ from the RMS calculation. First, we factor out $1/n$ as follows:

$$\text{RMS}(\vec{a}) = \sqrt{\frac{1}{n}\sum_{i=1}^{n} a_i^2} = \sqrt{\frac{1}{n}}\sqrt{\sum_{i=1}^{n} a_i^2} = \sqrt{\frac{1}{n}} \cdot \text{RSS}(\vec{a})$$

where $\text{RSS}(\vec{a}) = \sqrt{\sum_{i=1}^{n} a_i^2}$ is the Root Sum Square. We can now merge the constant term into the normalization weights g_i as follows:

$$y_i = \frac{a_i}{\text{RMS}(\vec{a})} \cdot g_i = \frac{a_i}{\text{RSS}(\vec{a})}\sqrt{n} \cdot g_i = \frac{a_i}{\text{RSS}(\vec{a})} \cdot g_i^*$$

with new normalization weights $g_i^* = \sqrt{n} \cdot g_i$. These new normalization weights can now be merged with the weights $\mathbf{W}$ of the following linear layer as shown in the previous sections. This optimization also applies for the case where we add an epsilon as detailed in the previous section. In this case, we factor out $1/n$ as follows:

$$\text{RMSe}(\vec{a}) = \sqrt{\epsilon + \frac{1}{n}\sum_{i=1}^{n} a_i^2} = \sqrt{\frac{1}{n}\left(n\epsilon + \sum_{i=1}^{n} a_i^2\right)} = \sqrt{\frac{1}{n}} \cdot \text{RSSe}(\vec{a})$$

where $\text{RSSe}(\vec{a}) = \sqrt{n\epsilon + \sum_{i=1}^{n} a_i^2}$.

F RMSNorm Decode-Time Fraction Across Precisions

Setup. `Llama-3.2-1B-Instruct` loaded in HuggingFace Transformers on a single NVIDIA A100 (40 GB) at bf16, 8-bit BitsAndBytes, and 4-bit BitsAndBytes (NF4 with double quantization) [38]. For each precision, 20 trials of greedy decode generating 256 tokens after 3 warmup runs. CUDA-event hooks attached to every `LlamaRMSNorm` module record per-call start/end timestamps; cumulative RMSNorm time per generation is summed across all 33 norm calls per forward pass and reported alongside externally-measured total decode wall-clock.

Findings. RMSNorm occupies 24.62% of decode wall-clock at bf16, an order of magnitude above its FLOP share, because the model runs 33 separate RMSNorm kernels per forward pass and each incurs a fixed kernel-launch latency on the order of 10 μs plus a memory-bandwidth pass over the full activation tensor independent of the small RMS arithmetic. Roughly half of this fraction is recoverable today via a runtime swap of `LlamaRMSNorm.forward`: on the same hardware, same decode protocol, replacing it with PyTorch's existing weightless C++ kernel `F.rms_norm(x, shape, weight=None, eps)` yields a $+\mathbf{12.77}\%$ end-to-end speedup on the FlashNorm-folded

Table 5: RMSNorm wall-clock fraction during 256-token greedy decode on `Llama-3.2-1B-Instruct`, A100, bf16 inputs, median of 20 trials. Norm time is cumulative across all per-layer RMSNorm calls.

Precision	Tokens/s	Total decode (ms)	Norm time (ms)	Norm fraction
bf16	38.5	6,650	1,640	**24.62%**
8-bit (BitsAndBytes)	12.5	20,455	1,675	8.19%
4-bit (NF4 + DQ)	14.5	17,720	1,650	9.31%

variant `open-machine/Llama-3.2-1B-FlashNorm` (50.75 vs 45.00 tokens/s). The cumulative norm wall-clock is invariant under weight quantization (within $\pm 2\%$ across all three precisions: 1,640, 1,675, 1,650 ms), as BitsAndBytes does not quantize norm weights. The norm fraction is non-monotonic across precisions on this hardware ($24.62\% \rightarrow 8.19\% \rightarrow 9.31\%$) because BitsAndBytes inflates the linear path's wall-clock at low bit on A100 (`MatMul8bitLt` dequantize overhead makes 8-bit $3\times$ slower than bf16; NF4 is more efficient but still $2.7\times$ slower); the fraction shrinks at lower bit not because the norm becomes cheaper but because the BitsAndBytes linear becomes more expensive. On hardware-and-framework combinations with native low-bit kernels (e.g., `llama.cpp` GGUF, MLX 4-bit on Apple silicon, vLLM and TensorRT-LLM with int4 at scale), the invariant norm cost would translate to a monotonically growing fraction; direct measurement on those stacks is left to follow-up.

G Fused Kernel on A100: Full Benchmark and Two-Source Decomposition

This appendix provides the full benchmark results and a quantitative decomposition supporting the A100 fused-kernel prototype summarized in Section A.1. The prototype is a pair of Triton [39] kernels: a scalar-op decode kernel used when both the token count M and the hidden dimension K are small, and an autotuned tensor-core GEMM kernel used otherwise.

G.1 Two-source decomposition

FlashNorm's speedup arises from two mathematically independent sources.

Source A: Weight-fold savings. Folding $\mathbf{g}$ into $\mathbf{W}^*$ (Proposition 1) eliminates the elementwise multiply by $\mathbf{g}$ inside the RMSNorm kernel. This source is architecture-agnostic and applies at any model scale. Source A requires no kernel engineering beyond a one-time checkpoint preprocessing step. Its absolute saving is small but always positive.

Source B: Kernel-fusion savings. Merging RMSNorm with the subsequent matmul into a single kernel eliminates three per-call overheads that scale with the number of kernel invocations rather than with matmul compute: (i) one kernel launch, approximately 10 to 15 μs on A100; (ii) one intermediate tensor allocation for the normalized activations, approximately 5 μs; and (iii) one HBM round-trip on the normalized activations. The combined saving is approximately 15 to 25 μs per call on A100, a fixed cost that does not scale with model size.

The GEMM-quality interaction. Realizing Source B requires implementing the fused kernel in a GPU DSL whose matmul codegen is competitive with vendor-tuned cuBLAS. Our Triton prototype produces a matmul running at approximately 75 to 85% of cuBLAS's compute-bound throughput, a fixed percentage cost that scales with matmul time. At small model scales the fixed Source-B saving exceeds this matmul penalty and the prototype wins; at larger model scales the penalty dominates and the prototype loses. This is a limitation of Triton's matmul codegen at compute-bound shapes, not of the FlashNorm transformation itself. A production implementation backed by CUTLASS or hand-tuned tensor-core-intrinsic CUDA, which is the kernel infrastructure already used by production inference frameworks such as vLLM [28] and SGLang [29], would eliminate this percentage penalty and preserve Source B across scales.

G.2 Full benchmark results

Table 6 reports the complete 18-row benchmark across three model scales and six token counts. All times are milliseconds, averaged over 100 iterations after 20 warmup iterations. Inputs are fp16, accumulators are fp32, eps = 10^{-6}. The Path column shows which Triton kernel was dispatched: the decode kernel when $M < 128$ and $K < 1024$, the GEMM kernel otherwise.

Table 6: Full A100 benchmark for the fused Triton `RMSNorm+QKV` prototype. Shapes (n, k): (576, 960) for SmolLM2-135M, (2048, 2560) for Llama-3.2-1B, (4096, 6144) for Llama-3.1-8B. Speedup columns are computed as $(t_{\text{baseline}} - t_{\text{FlashNorm}})/t_{\text{baseline}}$.

Model	Tokens	Path	PyTorch	vLLM	FlashNorm	vs PyTorch	vs vLLM
SmolLM2-135M	1	decode	0.054	0.052	0.040	+24.8%	+22.2%
SmolLM2-135M	16	decode	0.056	0.054	0.039	+29.1%	+26.5%
SmolLM2-135M	64	decode	0.084	0.080	0.054	+35.8%	+32.6%
SmolLM2-135M	256	GEMM	0.060	0.059	0.069	−15.2%	−16.9%
SmolLM2-135M	1024	GEMM	0.059	0.057	0.060	−2.7%	−6.0%
SmolLM2-135M	4096	GEMM	0.072	0.066	0.061	+14.5%	+7.7%
Llama-3.2-1B	1	GEMM	0.053	0.051	0.061	−14.4%	−19.5%
Llama-3.2-1B	16	GEMM	0.067	0.065	0.062	+7.2%	+4.7%
Llama-3.2-1B	64	GEMM	0.055	0.051	0.061	−10.8%	−18.5%
Llama-3.2-1B	256	GEMM	0.056	0.053	0.061	−8.0%	−14.5%
Llama-3.2-1B	1024	GEMM	0.063	0.062	0.094	−48.5%	−51.6%
Llama-3.2-1B	4096	GEMM	0.195	0.198	0.294	−50.8%	−48.5%
Llama-3.1-8B	1	GEMM	0.063	0.057	0.073	−17.0%	−29.3%
Llama-3.1-8B	16	GEMM	0.062	0.057	0.069	−12.1%	−21.4%
Llama-3.1-8B	64	GEMM	0.057	0.053	0.069	−21.3%	−29.3%
Llama-3.1-8B	256	GEMM	0.076	0.072	0.105	−38.0%	−45.1%
Llama-3.1-8B	1024	GEMM	0.225	0.233	0.371	−64.8%	−59.1%
Llama-3.1-8B	4096	GEMM	0.908	0.924	1.354	−49.1%	−46.5%

G.3 Interpretation and production path

The SmolLM2-135M rows constitute six positive data points against both production baselines: three decode-kernel wins at $M \in \{1, 16, 64\}$ (**+22.2 to +35.8%**) and one GEMM-kernel win at M=4096 (**+7.7 to +14.5%**). These rows empirically validate Source B at this scale: the fixed fusion savings exceed the Triton-vs-cuBLAS GEMM-quality penalty.

The Llama-3.2-1B and Llama-3.1-8B rows are dominated by the Triton GEMM-quality penalty, which scales with matmul compute. The largest losses appear at compute-bound shapes ($M \geq 1024$), where the percentage cost of Triton's matmul codegen relative to cuBLAS is largest in absolute terms. These losses are consistent with, and quantitatively predicted by, the two-source decomposition above. They do not reflect a limitation of the FlashNorm transformation, which is mathematically exact (Propositions 1 and 2) and applies identically at every model scale.

For production deployment, inference frameworks such as vLLM [28], SGLang [29], and TensorRT-LLM maintain CUTLASS- or cuBLAS-backed fused kernels that match vendor GEMM throughput on compute-bound shapes. Integrating FlashNorm's fusion pattern (Figures 1(c) and 8(c)) into such a stack would preserve Source B's fixed savings while incurring no matmul regression, extending the A100 speedup observed on SmolLM2 to larger model scales. We therefore position this prototype as a validation of the fusion hypothesis and an open invitation for production kernel integration, rather than a general speedup claim.

A CUTLASS-backed fused `RMSNorm+linear` kernel targeting vLLM integration is under active development, directly extending the dispatch and numerical design of the Triton prototype reported here. The CUTLASS implementation replaces the Triton matmul codegen with a vendor-grade GEMM while retaining the fused RMS accumulation in the mainloop and the deferred-normalization epilogue; this is the implementation tier expected to preserve Source B savings at Llama-family production scales.

As a prerequisite, we verified that an unfused CUTLASS GEMM already attains 92–111% of cuBLAS throughput at the Llama-family compute-bound prefill shapes used in this paper (specifically: 92.0% at Llama-3.2-1B M=1024, 98.3% at M=4096; 95.1% at Llama-3.1-8B M=1024, 97.8% at

M=4096; 111.4% at SmolLM2-135M M=1024), using the default Sm80 tile configuration without further tuning. Since Source B contributes an approximately fixed 15–25 μs saving per call on top of the matmul, this matmul-quality floor supports the expectation that adding FlashNorm fusion to a CUTLASS GEMM yields net speedups at these scales.

G.4 End-to-end validation: fused-kernel decode wins on A100

A hand-rolled Sm80 kernel using `nvcuda::wmma` tensor-core intrinsics with a decode-specialized tile (TB_M=16), a `TB_K`= 32 mainloop chunk, and a 3-stage `cp.async` pipeline demonstrates that the fusion savings described above materialize as measurable end-to-end wins at the decode regime (small M) across all three model scales used in this paper. Table 7 reports the headline results.

Table 7: End-to-end fused FlashNorm kernel vs realistic baseline (PyTorch native `rms_norm` followed by cuBLAS matmul) at decode token counts, on NVIDIA A100, fp16 inputs. The fused kernel uses a 3-stage `cp.async` pipeline with `TB_K`= 32 to fully hide HBM load latency behind tensor-core compute.

Model	Tokens	realistic	fused kernel	vs realistic
SmolLM2-135M	1	0.060 ms	0.017 ms	+72.3%
SmolLM2-135M	16	0.054 ms	0.017 ms	+69.1%
Llama-3.2-1B	1	0.079 ms	0.049 ms	+37.9%
Llama-3.2-1B	16	0.064 ms	0.048 ms	+24.7%
Llama-3.1-8B	1	0.068 ms	0.074 ms	−9.6% (parity)
Llama-3.1-8B	16	0.081 ms	0.074 ms	+8.5%

Five of six decode data points (three model scales across $M \in \{1, 16\}$) deliver positive speedup, with the remaining Llama-3.1-8B M=1 row at parity within measurement noise, confirming that the fusion hypothesis translates to wall-clock wins at the token counts most relevant to production inference. The gap at Llama-3.1-8B M=1 is attributable to cuBLAS's specialized gemv heuristic path which our uniform-warp kernel cannot match without warp-specialized producer/consumer scheduling. A CUTLASS-based integration in production inference stacks, with warp specialization already present in their kernel infrastructure, would close this remaining gap.

G.5 H100 validation: CUTLASS 3.x Sm90 production kernel

To verify that the fusion pattern transfers to production-grade Sm90 infrastructure, we extend the A100 prototype to a CUTLASS 3.x Sm90 kernel built from the `CollectiveBuilder` and Epilogue Visitor Tree (EVT) templates NVIDIA ships for Hopper. The kernel uses TMA for HBM-to-shared bulk loads, WGMMA for 64×128×16 cooperative tensor-core matmul, and the hardware `mbarrier` primitive for producer-consumer warpgroup synchronization. A small pre-pass kernel computes $\mathrm{inv_rms}[m]$ and both kernels are captured into a single CUDA Graph, collapsing the host-visible dispatch to one `cudaGraphLaunch` per call. The per-row scaling is fused into the GEMM epilogue via the `Sm90ColBroadcast` EVT node. We measure against two realistic baselines: the HuggingFace Transformers path (`torch.nn.functional.rms_norm` followed by cuBLAS matmul), and an optimized path using `flashinfer.norm.rmsnorm` [30] (the fused RMS kernel vLLM and SGLang use internally) followed by cuBLAS matmul. Table 8 reports the full 18-shape benchmark.

All 18 shapes deliver positive speedup versus the HuggingFace path (+28% to +82%). Against the optimized FlashInfer plus cuBLAS baseline, V12 wins on 10 of 18 shapes: all SmolLM2-135M scales (+42% to +52%) and Llama-3.2-1B decode ($M \leq 256$, +17% to +23%). V12 loses on 8 shapes: Llama-3.2-1B compute-bound ($M \geq 1024$, −33% to −45%) and all Llama-3.1-8B rows (−20% to −66%). The honest interpretation follows from the two-source decomposition: at scales where Source B's 15–25 μs fusion saving is a meaningful fraction of call time (SmolLM2 and Llama-1B decode), V12 wins decisively against both baselines; at larger Llama-family shapes, flashinfer's fused RMS kernel is already fast enough (25–80 μs) that V12's fusion saving is smaller than the matmul penalty incurred by a non-specialized tile selection.

Where the losses come from. The 8 losing rows share a single root cause: V12 instantiates a single `CollectiveBuilder` variant with `TileShape`= $\langle 128, 128, 64 \rangle$, which is reasonable for prefill at small to medium scales but suboptimal in two regimes. First, at Llama-3.1-8B decode (M=1) the

Table 8: H100 fused FlashNorm kernel (CUTLASS 3.x Sm90 `CollectiveBuilder` with EVT, CUDA-graph captured) vs two realistic baselines: HuggingFace (`torch.nn.functional.rms_norm` plus cuBLAS) and FlashInfer (`flashinfer.norm.rmsnorm` plus cuBLAS). fp16 inputs, averaged over 100 iterations after 20 warmup. Shapes (n, k) as in Table 6.

Model	Tokens	HF (ms)	FlashInfer (ms)	V12 (ms)	vs HF	vs FlashInfer
SmolLM2-135M	1	0.096	0.038	0.019	+80.4%	+50.6%
SmolLM2-135M	16	0.096	0.039	0.019	+80.4%	+51.3%
SmolLM2-135M	64	0.097	0.038	0.019	+80.6%	+50.6%
SmolLM2-135M	256	0.097	0.039	0.019	+80.5%	+51.3%
SmolLM2-135M	1024	0.089	0.034	0.016	+82.0%	+52.2%
SmolLM2-135M	4096	0.079	0.035	0.021	+74.1%	+42.0%
Llama-3.2-1B	1	0.071	0.027	0.021	+70.9%	+23.3%
Llama-3.2-1B	16	0.065	0.026	0.021	+68.0%	+18.7%
Llama-3.2-1B	64	0.066	0.025	0.021	+68.2%	+17.6%
Llama-3.2-1B	256	0.065	0.026	0.021	+67.5%	+17.1%
Llama-3.2-1B	1024	0.072	0.026	0.037	+48.5%	−44.8%
Llama-3.2-1B	4096	0.221	0.075	0.100	+54.7%	−32.8%
Llama-3.1-8B	1	0.065	0.026	0.043	+34.2%	−65.7%
Llama-3.1-8B	16	0.065	0.026	0.040	+38.5%	−54.1%
Llama-3.1-8B	64	0.065	0.027	0.039	+39.7%	−47.8%
Llama-3.1-8B	256	0.074	0.032	0.040	+45.8%	−24.8%
Llama-3.1-8B	1024	0.150	0.079	0.096	+36.4%	−20.3%
Llama-3.1-8B	4096	0.565	0.293	0.405	+28.3%	−38.1%

tile pads to 128 rows while only one row carries real data, wasting $127/128$ of the tensor-core work; cuBLAS has a specialized gemv heuristic that does not incur this padding. Second, at Llama-family compute-bound shapes cuBLAS selects a differently shaped tile (for instance 256×128) that achieves higher SM occupancy and cache efficiency than our 128×128 default. In both regimes the fusion saving is real but is smaller than the matmul-quality gap from a non-specialized tile. This mirrors exactly the two-source decomposition of Appendix G: fusion savings are a fixed per-call quantity, and whether they yield a net win depends on how close the fused matmul runs to the vendor-optimal matmul at that specific shape.

Production path to universal wins. Closing the Llama-family gap requires two additions within the same CUTLASS 3.x framework, both standard kernel-team work inside production inference stacks.

(i) Shape-specialized tile dispatch. Multiple `CollectiveBuilder` instantiations with different `TileShape` variants (e.g. $\langle 16, 128, 64\rangle$ for decode, $\langle 128, 128, 64\rangle$ for transitional, $\langle 256, 128, 64\rangle$ for compute-bound prefill), dispatched by (M, K, N) at runtime. This is the same pattern as the Triton prototype's two-condition dispatch (Section G) and as cuBLAS's own internal heuristic. Engineering effort, approximately one week for a CUTLASS-familiar engineer.

(ii) Custom `CollectiveMainloop` with in-line RMS accumulation. Fuse $\sum_k x_{m,k}^2$ into the matmul's producer warpgroup during the existing tile-load phase, removing the pre-pass kernel entirely. The mainloop reads x from HBM regardless; accumulating x^2 costs register operations, not additional memory traffic. CUTLASS 3.x supports this via template extension of `CollectiveMmaFma`; a 400-line reference design accompanies the prototype notebook. Engineering effort, approximately two to four weeks.

With these two additions in place, the kernel would extend the measured small-scale wins of Table 8 to Llama-family production scales. Both additions sit within vLLM's [28] existing CUTLASS infrastructure, which routinely maintains shape-specific tile dispatch for its matmul fast paths. This is the engineering scope the paper positions as the natural next step for production integration.

End-to-end production throughput. The measurements in Table 8 are operation-level times for a single fused $\mathrm{RMSNorm} + \mathrm{linear}$ call. A complete end-to-end picture requires integrating the tile-dispatched fused op into the inference server's model execution graph at both the `input_layernorm`→`qkv_proj` and `post_attention_layernorm`→`gate_up_proj` dispatch

sites, then measuring the tokens-per-second impact on representative serving workloads. This paper establishes the mathematics (Propositions 1 and 2), the quality preservation across scales (Table 1), and the measured operation-level validation on T4, A100, and H100. The end-to-end throughput measurement belongs to the production integration that this work is designed to enable.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction state the two algebraic propositions, the preservation of model outputs, the three-model verification scope (SmolLM2-135M, Llama-3.2-1B, Llama-3.1-8B), and the T4 / A100 / H100 empirical scope. The reported speedups (33-35% T4 at SmolLM2 scale, 12-14% T4 at Llama-7B scale, 13% realized HF end-to-end on Llama-3.2-1B at bf16) match the experiments in Section 5 and Appendix app:rmsnorm-fraction.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 6 (Discussion and Limitations) addresses the RMSNorm-only scope, activation dynamic range under fixed-point backends, adaptive-dispatch fallback at small token counts, the prototype-kernel scope of the A100/H100 implementations, and the absence of direct measurements on native low-bit hardware.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: Propositions 1 and 3 are stated with their full assumptions (Section 3) and accompanied by short proofs (matrix associativity for the fold; scalar commutativity for the deferred normalization). Both proofs are elementary and contained in the main body.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Each experimental claim is accompanied by a publicly hosted reproducibility notebook (flashNorm_decode_profile.ipynb, flashNorm_hf_a100.ipynb, flashNorm_triton_kernel.ipynb, flashNorm_cutlass_kernel.ipynb) in the transformer-tricks repository. Hardware, dtype, protocol, and seed are pinned in each notebook.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The transformer-tricks repository contains the paper source, the flashify_repo() implementation in the tricks PyPI package, and the four reproducibility notebooks. Flashified model checkpoints are released under the weightless-rmsnorm HuggingFace collection. For anonymous review, an anonymised mirror is provided.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Each experiment specifies hardware (T4 / A100 40GB / H100), precision (bf16 / fp16 / 8-bit BitsAndBytes / NF4+DQ), token-count protocol, number of trials (n=20), and aggregation (median). BitsAndBytes 8-bit and 4-bit hyperparameters are listed in Appendix app:rmsnorm-fraction.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Decode-profile and realized-speedup measurements report medians over n=20 trials; T4 norm-then-project measurements report adaptive-dispatch deltas. Trial-to-trial variability on the Colab A100 is dominated by external scheduling jitter; relative rankings are stable across trials.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: T4 (Section 5.2), Colab A100 40GB (Appendices app:rmsnorm-fraction and app:a100), and H100 (Appendix app:h100-cutlass). Per-experiment runtime is short (each notebook runs in under thirty minutes); total reported compute is well under one GPU-day. Exploratory and failed runs were a small multiple of the reported compute.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: [Yes]

Justification: We have read the NeurIPS Code of Ethics; this work conforms to it.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.

- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: FlashNorm is foundational efficiency work that preserves model outputs bit-exactly; it does not change model capabilities and has no specific dual-use path. Lower inference cost may broaden access and reduce the energy footprint of LLM deployment.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: No new models with novel capabilities are released. The flashified checkpoints are bit-equivalent reformulations of existing public weights and introduce no new misuse surface.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: All third-party assets are cited with their version: PyTorch (BSD), HuggingFace Transformers (Apache 2.0), vLLM (Apache 2.0), llama.cpp (MIT), CUTLASS (BSD-3-Clause), Triton (MIT), bitsandbytes (MIT). Models: Llama-3.2-1B-Instruct (Llama 3.2 Community License), SmolLM2-135M (Apache 2.0), Llama-3.1-8B (Llama 3.1 Community License). Each is referenced in the bibliography.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The weightless-rmsnorm HuggingFace collection releases flashified checkpoints with model cards documenting the flashify recipe and license inheritance from the base models. The tricks PyPI package documents flashify_repo(). The repository README documents reproducibility notebooks.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: This work does not involve human subjects or crowdsourcing.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.

- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: No human subjects research is involved.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [Yes]

    Justification: Large language models (specifically Claude by Anthropic) were used as a coding and writing assistant during preparation of this work. Their role was limited to writing, editing, and code-drafting tasks: drafting reproducibility notebook scaffolds, editing prose, drafting upstream pull-request and issue bodies, and assisting with documentation. The core scientific contributions of the paper, namely the two algebraic propositions and their proofs (Section 3), the experimental design, the kernel implementations (Appendices C and D), and the empirical measurements, were developed and verified by the human authors. No LLM was involved in generating measurements, running benchmarks, or making methodological decisions. All quantitative results were independently reproduced by the authors before inclusion.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.